AI702 Lab 0 In-lab

Computing environment, arrays and experimental evidence

Implement array and tensor operations, inspect automatic differentiation and audit the reported accuracy of a supplied classifier on Breast Cancer Wisconsin data.

Getting started

Work from the folder containing lab.py. Open a terminal there and create the course environment from the supplied lockfile:

```
uv sync --locked
uv run --locked python lab.py check 0
```

Use Python 3.12 through uv. Do not regenerate uv.lock or change dependency versions. Show an environment or verification error to the TA before continuing should there be any.

The lab pack

Each lab has its own folder. The functions to implement are in student.py. Data loading, public tests, experiment runners and plotting routines are supplied.

Location	Purpose and permitted use
labN/student.py	Implement the assigned functions and classes. Preserve their interfaces.
labN/settings.json	Use the configuration assigned or approved by the TA.
labN/notes.md	Record predictions, measurements, explanations and any permitted help.
labN/run.py and test_labN.py	Read the supplied runner and public tests. **Do not edit them.**
labN/results/	Results files and HTML reports. **Do not edit these files.**
labN/submissions/	Submission ZIPs created by the launcher. **Do not alter them.**
lab.py and shared/	Supplied code for running experiments. **Do not modify it.**

Only student.py, settings.json and notes.md are editable for this assignment. Keep the supplied tests, configuration infrastructure, policy and verification files unchanged.

Command reference

Run every command from the folder containing lab.py. Replace N with the lab number and RUN_ID with the ID of a completed run.

Command after uv run --locked python lab.py	Purpose
check N	Check the supplied files and environment.
test N	Run the public tests.
run N	Run the main experiment.
run N --smoke	Run a small diagnostic that cannot be submitted for assessment.
view N	Open the latest main experiment report.
view N --run-id RUN_ID	Open the report for the chosen main experiment.

Command after uv run --locked python lab.py	Purpose
view N --no-open	Print the report path without opening a browser.
transfer N	Display and verify the additional task settings.
run N --stage transfer	Run the additional experiment.
package N --run-id RUN_ID	Create a ZIP with the chosen main experiment and final notes.
--help	Display commands. Use run --help for run options.

The commands call the main experiment core and the additional task transfer. Use --stage transfer with view and package for the additional task. Without --run-id, they select the latest completed run in that stage. **Always specify the run ID when creating a submission ZIP.** The baseline 0 demonstration cannot be submitted for assessment.

Use CPU unless the TA approves another device. The following command prepares the dataset:

```
uv run --locked python lab.py data breast_cancer
```

The data cache is ~/.cache/ai702 unless another location is specified. Keep data and generated outputs separate from your implementation files.

Resources

Follow AI_POLICY.md. Use only the approved course resources and TA assistance. Disable AI completion and coding assistants. Do not use conversational AI or agents to implement, debug, review or explain assessed work. Record any permitted help in notes.md. **Do not modify supplied tests, substitute data or edit results files.**

An initial NotImplementedError means a function is not finished. Read its instructions and the public test to work out what is missing. Ask the TA about environment, verification or download errors.

Problem 1 Inspect the supplied experiment

Run the demonstration before completing your implementation:

```
uv run --locked python lab.py baseline 0
uv run --locked python lab.py view 0 --stage baseline
```

(a) Locate metrics.json, validation_predictions.csv and run.json through the report. Identify the target column, prediction column and sample identifiers from validation_predictions.csv. State which columns in validation_predictions.csv would let you recompute the metric without retraining the classifier.

(b) Inspect the reported accuracy and its stated unit. In lab0/notes.md, describe how you will do a sanity check and what you expect to find **before calculating a replacement value**. Submit the answers to these questions as part of your notes.md.

Problem 2 Implement the array and tensor interfaces

(a) Complete select_columns, standardize_columns, linear_scores_numpy and LinearTransform.

- In LinearTransform, preserve the batch dimension assuming the input weights are of shape (output_hidden_dim, input_hidden_dim).

- Your standardization implementation must use the supplied statistics, and leave its inputs unchanged whenever a zero standard deviation is present in the array of standard deviations in the input. The class must produce the same output as the NumPy function for standardization.

(b) Complete torch_linear_scores, linear_scores_numpy, and rearrange_nchw_to_nhwc. Return a contiguous NHWC tensor from the rearrangement. Explain why changing the order of dimensions is different from simply reshaping the tensor so that its dimensions match the order of dimensions rearranged tensor. Put your answer in notes.md.

Run the public tests, then the complete experiment:

```
uv run --locked python lab.py test 0
uv run --locked python lab.py run 0
uv run --locked python lab.py view 0
```

(d) Use details.json and raw.npz to compare NumPy and PyTorch scores to inspect the memory-layout demonstration. Recompute validation accuracy by computing the matching rates of rows from all CSV rows. Give both the value on the declared scale and the corresponding percentage, then identify the exact inconsistency in the supplied report.

Checkpoint. Show the TA the shapes through one calculation, including the batch dimension, the NumPy/PyTorch score comparison and the rows you used to check the accuracy unit. Be prepared to explain one of your functions on a new input.

Inspect and submit your results

Each run is saved in labN/results/STAGE/RUN_ID/ with its starting code, settings and notes, its measurements and index.html. Open JSON and CSV files without saving changes. NPZ files contain named NumPy arrays. To inspect your main Lab 0 results, start Python from the folder containing lab.py:

```
uv run --locked python
```

At the Python prompt, enter the following. Replace RUN_ID with the ID of your main experiment:

```
import numpy as np
archive = np.load("lab0/results/core/RUN_ID/raw.npz")
archive.files
archive["predictions"].shape
archive.close()
exit()
```

While the file is open, use the name of one of the arrays from archive.files to inspect its values. Upcoming labs' NPZ files work the same way. Record run IDs and result paths in notes.md. Include sample IDs when discussing individual predictions.

Finish your observations in notes.md and record any help you received. Then create the submission ZIP for your main experiment:

```
uv run --locked python lab.py package 0 --run-id RUN_ID
```

Submit the ZIP and requested IDs in the LMS. Keep the original results and ZIP. After changing code or settings, run the experiment again and create a new ZIP before the deadline. **Do not overwrite an existing ZIP.**

Problem 3 Additional experiment

Use the assigned transfer.json and transfer.sig in lab0/. **Do not modify either file.** Read the assigned settings and write down what you expect to happen **before running the experiment**:

```
uv run --locked python lab.py transfer 0
uv run --locked python lab.py run 0 --stage transfer
uv run --locked python lab.py view 0 --stage transfer
```

The launcher applies the assigned settings automatically. Compare the new results with your main experiment and explain the differences in notes.md. Create a separate submission ZIP for this run:

```
uv run --locked python lab.py package 0 --stage transfer --run-id RUN_ID
```

Submit this ZIP and the IDs requested in the LMS. Keep both submission ZIPs.